

软件详细设计说明书

智慧烹饪系统—智能加热炉 APP

版本号: V1.0

日期: 2026 年 6 月 28 日

1. 引言

1.1 编写目的

本文档在概要设计的基础上,对智慧烹饪系统的各个模块进行详细设计,给出各模块的实现逻辑、算法、数据结构、接口细节等内容,为编码实现提供直接依据。

1.2 参考资料

- 《软件需求规格说明书 V1.0》
 - 《软件概要设计说明书 V1.0》
 - 《12KW 电磁机芯串口通信协议 v2.0》
 - 《uni-app 开发文档》
-

2. 蓝牙通信模块详细设计 (ble.js)

2.1 模块概述

蓝牙通信模块是系统的核心底层模块,封装了所有 BLE 蓝牙相关操作,向上层页面提供统一的指令发送和消息接收接口。模块采用单例模式设计,全局唯一实例 bleManager。

2.2 常量定义

2.2.1 UUID 常量

```
const SERVICE_UUID = '0000FFB0-0000-1000-8000-00805F9B34FB'
const WRITE_UUID = '0000FFB2-0000-1000-8000-00805F9B34FB'
const NOTIFY_UUID = '0000FFB1-0000-1000-8000-00805F9B34FB'
```

2.2.2 指令码常量 (CMD)

```
const CMD = {
  POWER_ON: '>AFO',
  POWER_OFF: '>AFF',
  GEAR_1: '>AF1', GEAR_2: '>AF2', GEAR_3: '>AF3', GEAR_4: '>AF4',
  GEAR_5: '>AF5', GEAR_6: '>AF6', GEAR_7: '>AF7', GEAR_8: '>AF8',
  GEAR_9: '>AF9', GEAR_10: '>AFA', GEAR_11: '>AFB', GEAR_12: '>AFC',
  ZONE_INNER: '>ACI',
  ZONE_OUTER: '>ACX',
  ZONE_ALL: '>ACL',
  INNER_SEG_1: '>AA1', INNER_SEG_2: '>AA2', INNER_SEG_3: '>AA3',
  INNER_SEG_4: '>AA4', INNER_SEG_5: '>AA5',
  OUTER_SEG_1: '>AB1', OUTER_SEG_2: '>AB2', OUTER_SEG_3: '>AB3',
  OUTER_SEG_4: '>AB4', OUTER_SEG_5: '>AB5',
```

```

    QUERY_TEMP: '>AT?',
    QUERY_POWER: '>AP?',
    ACK_OK: '>OK'
  }
}

```

2.2.3 异常描述映射

```

const ALARM_DESCS = {
  '>AE1': '机芯超温',
  '>AE2': '线盘超温',
  '>AE3': '过流保护',
  '>AE4': '电流异常',
  '>AE5': '锅具异常',
  '>AE6': '线盘测温开路'
}

```

2.3 bleManager 对象设计

2.3.1 内部状态

状态变量	类型	初始值	说明
deviceId	String	''	当前连接设备 ID
connected	Boolean	false	是否已连接
_messageCallback	Function	null	消息回调函数

2.3.2 指令发送方法 powerOn() — 开机

功能：发送开机指令

参数：无

返回：Promise

流程：

1. 调用 _sendCommand(CMD.POWER_ON)
2. 返回 Promise

异常：连接断开时 reject

powerOff() — 关机

功能：发送关机指令

参数：无

返回：Promise

流程：

1. 调用 _sendCommand(CMD.POWER_OFF)
2. 返回 Promise

setGear(level) — 设置档位

功能：设置火力档位

参数：level (Number) — 档位 1~12

返回：Promise

流程：

1. 参数校验：level 必须在 1~12 之间
2. 映射指令码：
 - 1~9 → >AF1 ~ >AF9
 - 10 → >AFA
 - 11 → >AFB

- 12 → >AFC
- 3. 调用 `_sendCommand(cmd)`
- 4. 返回 `Promise`

setZone(mode) —设置分区模式

功能：设置内外分区整体模式

参数：mode (String) — 'inner' | 'outer' | 'all'

返回：Promise

流程：

1. 根据 mode 映射指令：
 - inner → >ACI
 - outer → >ACX
 - all → >ACL
2. 调用 `_sendCommand(cmd)`

setInnerSegment(seg) / setOuterSegment(seg) —分段控制

功能：设置内区/外区分段

参数：seg (Number) — 1~5

返回：Promise

流程：

1. 映射指令：内区 >AA1~5, 外区 >AB1~5
2. 调用 `_sendCommand(cmd)`

queryTemperature() —查询温度

功能：查询三路温度

参数：无

返回：Promise

流程：

1. 发送 >AT?
2. 设备返回 >A + 12位十六进制
3. 由消息监听回调解析

queryPower() —查询电参数

功能：查询电压/功率/电流

参数：无

返回：Promise

流程：

1. 发送 >AP?
2. 设备返回 >A + 12位十六进制
3. 由消息监听回调解析

2.3.3 内部发送方法 `**_sendCommand(cmdStr)**`

功能：底层指令发送

参数：cmdStr (String) — 完整指令字符串

返回：Promise

流程：

1. 检查连接状态，未连接则 reject
2. 将字符串转为 ArrayBuffer
3. 调用 `uni.writeBLECharacteristicValue`
4. 成功则 resolve, 失败则 reject

字符串转 ArrayBuffer 算法：

输入：ASCII 字符串

输出: Uint8Array

步骤:

1. 创建 `new Uint8Array(str.length)`
2. 遍历每个字符, 取 `charCodeAt(i)`
3. 返回 `buffer`

2.3.4 消息监听与解析 `onMessage(callback)`

功能: 注册设备消息回调

参数: `callback (Function)` — 回调函数, 参数为原始数据字符串

返回: 无

说明:

- 可注册多个回调 (数组存储)
- 收到设备消息时依次调用

消息监听主流程:

设备发送数据

```
uni.onBLECharacteristicValueChange
```

`ArrayBuffer` → 字符串 (ASCII解码)

遍历所有注册的回调

回调(`data`)

2.3.5 数据解析方法 `isAckOk(data)`

功能: 判断是否为成功应答

参数: `data (String)`

返回: `Boolean`

算法: `return data === '>OK'`

`isAlarm(data)`

功能: 判断是否为异常上报

参数: `data (String)`

返回: `Boolean`

算法: `return data in ALARM_DESCS`

`isQueryResponse(data)`

功能: 判断是否为查询响应

参数: `data (String)`

返回: `Boolean`

算法: `return data.startsWith('>A') && data.length === 13`

`parseTemperatureData(data)`

功能: 解析三路温度数据

参数: `data (String)` — 格式 `>A + 12位HEX`

返回: `{ channel1, channel2, channel3 }` 或 `null`

算法:

1. 去掉前缀 '>A', 取后续12个字符
2. 每4位一组:
 - 0-3位 → 1路温度 (十六进制转十进制)
 - 4-7位 → 2路温度
 - 8-11位 → 3路温度
3. 返回解析结果

parsePowerData(data)

功能: 解析电参数数据

参数: data (String) — 格式 >A + 12位HEX

返回: { voltage, power, current } 或 null

算法:

1. 去掉前缀 '>A'
2. 前4位 → 电压
3. 中间4位 → 功率
4. 后4位 → 电流
5. 十六进制转十进制返回

3. 首页模块详细设计 (index.vue)

3.1 模块结构

首页模块由以下部分组成: - 顶部导航栏 (标题 + 菜单按钮) - 下拉菜单 (创建设备组/添加设备/删除设备组) - 空状态展示 (无设备时) - 设备组列表 (可展开/收起) - 设备卡片网格 (两列布局) - 添加设备弹窗 (蓝牙扫描) - 拖拽浮动层 - 自定义底部导航栏

3.2 数据设计

3.2.1 data 数据

变量名	类型	初始值	说明
menuOpen	Boolean	false	下拉菜单是否展开
activeTab	Number	0	当前选中底部 Tab
deviceGroups	Array	[{name: '未分组设备', expanded:false, devices:[]}]	设备组列表
showAddDevice	Boolean	false	添加设备弹窗显示
bleScanning	Boolean	false	是否正在扫描蓝牙
bleSearched	Boolean	false	是否已完成扫描
bleDevices	Array	[]	扫描到的蓝牙设备列表
testDeviceCounter	Number	0	测试设备计数器
dragDevice	Object	null	当前拖拽的设备
dragSourceGroup	Object	null	拖拽源设备组
dragSourceIndex	Number	-1	拖拽源索引
dragOverGroup	Number	-1	拖拽目标组索引
isDragging	Boolean	false	是否正在拖拽
dragTimer	Object	null	长按计时器

3.2.2 计算属性 hasGroups

功能: 判断是否有任何设备或设备组

返回: Boolean

算法:

```
return deviceGroups.length > 1  
    || deviceGroups[0].devices.length > 0
```

dragFloatStyle

功能: 计算拖拽浮动卡片的位置样式

返回: Object (CSS样式对象)

算法:

根据 dragCurrentX/Y 和 dragOffsetX/Y 计算
使用 translate3d 实现硬件加速

3.3 方法详细设计

3.3.1 设备组管理方法 onCreateGroup()

功能: 创建设备组

流程:

1. 关闭下拉菜单
2. 弹出输入框 (uni.showModal editable)
3. 用户确认且内容非空 →
插入到 deviceGroups 倒数第二位 (未分组设备始终在最后)
4. 显示"创建成功"

边界:

- 名称为空 → 提示"请输入组名称"
- 用户取消 → 无操作

onEditGroupName(gi)

功能: 编辑设备组名称

参数: gi — 组索引

流程:

1. 检查是否为最后一组 (未分组设备)
2. 是 → 提示不可修改
3. 否 → 弹出编辑框, 当前名为默认值
4. 确认后更新 group.name

onDeleteGroup()

功能: 删除设备组

流程:

1. 关闭菜单
2. 获取可删除组 (排除最后一个"未分组设备")
3. 无可删除组 → 提示
4. 有 → 弹出 ActionSheet 选择要删除的组
5. 选择后 → 确认对话框
6. 确认 → 组内设备移至最后一组, 删除该组

toggleGroup(gi)

功能: 展开/收起设备组

参数: gi — 组索引

算法: deviceGroups[gi].expanded = !deviceGroups[gi].expanded

3.3.2 蓝牙扫描与添加方法 startBleScan()

功能: 开始蓝牙设备扫描

流程:

1. 设置扫描状态，清空设备列表
2. `uni.openBluetoothAdapter` 初始化蓝牙
3. 成功 → `uni.startBluetoothDevicesDiscovery` 开始扫描
4. 注册 `uni.onBluetoothDeviceFound` 监听
 - 过滤 `RSSI < -100` 的设备
 - 按 MAC 地址去重
 - 加入 `bleDevices` 列表
5. 5秒后自动停止扫描
6. 失败处理：
 - 适配器失败 → "请开启手机蓝牙"
 - 扫描失败 → "蓝牙搜索失败"

stopBleScan()

功能：停止蓝牙扫描

流程：

1. `uni.stopBluetoothDevicesDiscovery`
2. 设置 `bleScanning = false`, `bleSearched = true`

selectBleDevice(dev)

功能：选择扫描到的设备并添加

参数：dev — 设备对象 {name, mac, rssi}

流程：

1. 创建设备对象


```
{ name: dev.name, icon: '', power: 0,
  statusText: '待机', offline: false,
  mac: dev.mac, deviceType: 'cooker' }
```
2. 添加到最后一组（未分组设备）
3. 关闭弹窗
4. Toast 提示

addTestDevice()

功能：添加测试设备（无需真实蓝牙）

流程：

1. `testDeviceCounter++`
2. 从4种类型循环选取：

测试灶具-A / 测试灶具-B / 测试烤炉-A / 测试烤炉-B
3. 命名：{类型}-{序号}
4. MAC：TEST:00:00:00:XX（补零）
5. 添加到未分组设备

3.3.3 拖拽排序方法 onDeviceTouchStart(e, device, group, gi, di)

功能：触摸开始，准备长按拖拽

流程：

1. 记录触摸起始坐标
2. 记录拖拽源信息（设备、组、索引）
3. 获取所有 `.section` 元素的位置（用于判断跨组）
4. 启动 1秒 定时器
5. 定时器到 → `dragReady = true`（进入拖拽模式）

onDeviceTouchMove(e)

功能：触摸移动，处理拖拽

流程：

1. 未进入拖拽模式 → 返回

2. 移动距离 > 20px → 标记 isDragging = true
3. 更新当前坐标
4. 根据纵向偏移量判断目标组：
 - 向上偏移 > 80px 且不是第一组 → 目标组 = 上一组
 - 向下偏移 > 80px 且不是最后组 → 目标组 = 下一组
 - 否则 → 当前组
5. 更新 dragOverGroup

onDeviceTouchEnd()

功能：触摸结束，完成拖拽

流程：

1. 清除长按定时器
2. 未拖拽 → 重置状态，返回
3. 有拖拽且目标组不同 →
 - 从源组 splice 出设备
 - push 到目标组
 - 提示"已移至xxx组"
4. 重置所有拖拽状态

3.3.4 批量控制方法 batchPowerOn(group) / batchPowerOff(group)

功能：组内设备批量开机/关机

参数：group — 设备组对象

流程：

1. 确认对话框
2. 确认 → 遍历组内设备
 - 在线设备 → 更新 power 和 statusText
 - 计数
3. Toast 显示操作结果

3.3.5 测试预警方法 showTestAlarm()

功能：显示测试预警选择

流程：

1. 收集所有设备
2. 无设备 → 提示"请先添加设备"
3. 有 → 弹出 ActionSheet 选择设备
4. 选择后 → 弹出 ActionSheet 选择异常类型（6种）
5. 选择后 → 调用 _triggerAlarm

****_triggerAlarm(device, alarm)****

功能：触发模拟报警

参数：device — 设备对象, alarm — {label, cmd}

流程：

1. 构造 alarmData 对象
2. 存入 storage.pendingAlarm (跨页传递)
3. 全局事件 uni.\$emit('deviceAlarm')
4. 设置当前选中设备
5. 跳转到控制页
6. Toast 提示

3.3.6 导航与页面事件 onShow() / onHide()

onShow: 注册 deviceAlarm 事件监听

onHide: 取消 deviceAlarm 事件监听

****_onDeviceAlarm(data)****

功能: 接收报警事件, 自动跳转

流程:

1. 查找匹配设备 (按 deviceId 匹配 mac 或 name)
2. 找到 → 设置选中设备, 跳转到控制页

4. 控制页模块详细设计 (detail.vue)

4.1 模块结构

控制页由以下部分组成: - 顶部导航栏 (返回 + 设备名 + 更多菜单) - 更多菜单 (重命名/删除) - 开关机控制区 - 火候控制区 - 定时任务区 - 异常警报区 - 指令日志区 - 自定义底部导航栏

4.2 数据设计

4.2.1 data 数据

变量名	类型	初始值	说明
deviceName	String	'智能动热灶'	设备名称
deviceId	String	''	设备唯一标识
online	Boolean	true	是否在线
heatLevel	Number	0	当前火力档位
menuOpen	Boolean	false	更多菜单开关
timerRunning	Boolean	false	定时是否运行中
timerPaused	Boolean	false	定时是否暂停
timerSeconds	Number	0	剩余秒数
timerTotal	Number	0	总秒数
timerInterval	Object	null	定时间隔器
alarms	Array	[]	报警列表
alarmVibrateInterval	Object	null	震动间隔器
logs	Array	[]	日志列表
zoneMode	String	'all'	分区模式
innerSegment	Number	0	内区分段
outerSegment	Number	0	外区分段

4.2.2 计算属性

计算属性	类型	说明
hasDevice	Boolean	是否有选中设备
isTestDevice	Boolean	是否为测试设备
timerDisplay	String	定时显示 (MM:SS)
heatStatusText	String	火力状态文本
isPoweredOn	Boolean	是否已开机 (档位 > 0)

4.3 方法详细设计

4.3.1 生命周期与初始化 onShow()

功能：页面显示时初始化

流程：

1. 从 storage 读取 selectedDeviceId
2. 有设备 →
 - 设置 deviceId, deviceName
 - 添加日志"进入控制页面"、"设备已连接"
 - 初始化蓝牙消息监听 _initBleListener
3. 无 → 清空设备信息
4. 注册 deviceAlarm 事件监听
5. 检查 pendingAlarm, 有则处理

onHide() / onUnload()

onHide: 取消 deviceAlarm 监听

onUnload: 清除定时器 + 长按 + 震动

4.3.2 蓝牙消息监听 **_initBleListener()**

功能：初始化蓝牙消息监听

流程：

1. 调用 bleManager.onMessage(callback)
2. 回调中：
 - a. isAckOk → 记录" 指令执行成功"
 - b. isAlarm →
 - 获取异常描述
 - 添加到报警列表 (unshift, 最新在前)
 - 记录日志
 - 全局事件通知
 - 启动震动 startAlarmVibrate
 - c. isQueryResponse →
 - 尝试解析温度 → 记录日志
 - 尝试解析电参数 → 记录日志

4.3.3 开关机控制 powerOn()

功能：开机

流程：

1. 离线检查 → 提示返回
2. 已开机 → 提示"设备已开机"
3. bleManager.powerOn()
 - catch → 非测试设备提示失败
4. heatLevel = 1
5. 记录日志"开机 [>AF0]"

powerOff()

功能：关机

流程：

1. 离线检查
2. bleManager.powerOff()
3. heatLevel = 0
4. 记录日志"关机 [>AFF]"

4.3.4 火候控制 **_sendGearCommand(level)**

功能：发送档位指令（内部方法）

参数: level (Number) — 0~12

返回: Promise

流程:

1. 离线 → reject
2. 档位 0 → 调用 powerOff
3. 档位 1~12 → 调用 bleManager.setGear(level)
4. catch → 非测试设备 Toast 提示

setHeat(level)

功能: 设置指定档位 (快捷火候)

参数: level (Number)

流程:

1. 离线检查
2. 更新 heatLevel
3. 调用 _sendGearCommand
4. 生成日志 (档位名称 + 指令码)
命名映射: 3→小火, 6→中火, 9→大火, 12→爆炒
指令映射: >AF{数字或字母}

increaseHeat() / decreaseHeat()

功能: 加减档

流程:

1. 离线检查
2. 在 0~12 范围内增减
3. 调用 _sendGearCommand
4. 记录日志

长按连续加减:

onPressDown(dir) — 按下

- 500ms 后启动 setInterval (200ms间隔)
- 持续调用 increase/decrease

onPressUp() — 抬起

- 清除定时器

4.3.5 分区控制 setZoneMode(mode)

功能: 设置分区模式

参数: mode — 'inner' | 'outer' | 'all'

流程:

1. 离线检查
2. 更新 zoneMode
3. bleManager.setZone(mode)
4. 记录日志 (描述 + 指令)

setInnerSegment(seg) / setOuterSegment(seg)

功能: 设置内/外区分段

参数: seg — 1~5

流程:

1. 离线检查
2. 更新 innerSegment / outerSegment
3. bleManager.setInner/OuterSegment(seg)
4. 记录日志

4.3.6 定时任务 addTimer()

功能：添加定时任务

流程：

1. 生成 1~60 分钟选项列表
2. ActionSheet 选择
3. 选择后 →
 - 设置总秒数 = 分钟 * 60
 - timerRunning = true, timerPaused = false
 - 启动 startTimer
 - Toast + 日志

startTimer()

功能：启动倒计时

流程：

1. 清除已有定时器
2. 创建 1秒 interval
3. 每秒：
 - timerSeconds--
 - 到 0 →
 - 清除定时器
 - 状态重置
 - bleManager.powerOff()
 - heatLevel = 0
 - Toast "时间到！已自动关机"
 - uni.vibrateLong()
 - 日志

toggleTimer()

功能：暂停/继续定时

流程：

- 暂停 → timerPaused = true, 清除定时器
- 继续 → timerPaused = false, 重启 startTimer

cancelTimer()

功能：取消定时任务

流程：

1. 清除定时器
2. 重置所有定时状态
3. 记录日志
4. Toast 提示

4.3.7 异常报警 **_onExternalAlarm(data)**

功能：处理外部传入的报警（从首页跳转）

参数：data — {alarmCmd, ...}

流程：

1. 校验 alarmCmd
2. 从 alarmList 映射中查找
3. 找到 →
 - 添加到 alarms 列表头部
 - 记录日志
 - 启动震动

startAlarmVibrate()

功能：启动持续震动提醒

流程：

1. 清除已有震动定时器
2. 立即震动一次 (vibrateLong)
3. 启动 500ms interval, 持续震动

clearAlarmVibrate()

功能：停止震动

算法：清除 interval, 置 null

shutdownAlarm(idx)

功能：报警确认关机

参数：idx — 报警索引

流程：

1. bleManager.powerOff()
2. heatLevel = 0
3. 从 alarms 中 splice 删除
4. 记录日志
5. Toast 提示
6. 无剩余报警 → 停止震动

simulateAlarm()

功能：页面内模拟报警（测试用）

流程：

1. 6种异常 ActionSheet
2. 选择 →
 - 添加到 alarms
 - 记录日志
 - 启动震动

4.3.8 查询功能 queryTemperature() / queryPower()

功能：查询温度/电参数

流程：

1. 离线检查
2. 调用 bleManager.queryTemperature/queryPower
3. 记录日志（查询指令）
4. 返回数据由 _initBleListener 中的解析逻辑处理

4.3.9 指令日志 addLog(msg)

功能：添加日志条目

参数：msg — 日志消息

流程：

1. 获取当前时间 HH:MM:SS
2. 构造 { time, msg }
3. unshift 到 logs 数组（最新在前）

4.3.10 设备管理 onRename()

功能：重命名设备

流程：

1. 关闭菜单
2. 弹出编辑框（当前名为默认值）

3. 确认 → 更新 deviceName
4. 记录日志

onDelete()

功能：删除设备

流程：

1. 关闭菜单
 2. 确认对话框
 3. 确认 →
 - 清除 storage.selectedDeviceId
 - 清空设备信息
 - 重置 heatLevel, logs, alarms
 - 清除定时器
 4. Toast 提示
-

5. 关键算法设计

5.1 指令字符串映射算法

档位号 → 指令码映射：

输入：level (1~12)

输出：指令字符串

算法：

```
if level <= 9:
    return '>AF' + level
else:
    // 10→A, 11→B, 12→C
    char = String.fromCharCode(55 + level) // 55 = '7'.charCodeAt()
    return '>AF' + char
```

验证：

```
10 → 55+10=65 → 'A' → >AFA
11 → 55+11=66 → 'B' → >AFB
12 → 55+12=67 → 'C' → >AFC
```

5.2 十六进制数据解析算法

输入：data = '>A123456789ABC' (13字符)

输出：{ ch1, ch2, ch3 }

算法：

```
hexStr = data.slice(2) // 去掉 '>A' → '123456789ABC'
ch1 = parseInt(hexStr.slice(0, 4), 16) // 0x1234 = 4660
ch2 = parseInt(hexStr.slice(4, 8), 16) // 0x5678 = 22136
ch3 = parseInt(hexStr.slice(8, 12), 16) // 0x9ABC = 39612
```

5.3 火候条分段着色算法

输入：当前档位 level (0~12), 当前段号 i (1~12)

输出：CSS 类名

算法:

```
if level === 0 or offline:
    return 'heat-seg-off'
if i > level:
    return 'heat-seg-off'
if level <= 4:
    return 'heat-seg-blue'      // 1-4档 蓝色
elif level <= 8:
    return 'heat-seg-orange'    // 5-8档 橙色
else:
    return 'heat-seg-red'       // 9-12档 红色
```

5.4 拖拽跨组判断算法

输入:

sourceGi — 源组索引
moveDiff — 纵向移动距离 (像素)
totalGroups — 总组数

输出: targetGi — 目标组索引

算法:

```
if moveDiff < -80 AND sourceGi > 0:
    return sourceGi - 1      // 向上移动, 进入上一组
elif moveDiff > 80 AND sourceGi < totalGroups - 1:
    return sourceGi + 1      // 向下移动, 进入下一组
else:
    return sourceGi          // 还在当前组
```

5.5 时间格式化算法

输入: 总秒数 totalSeconds

输出: MM:SS 格式字符串

算法:

```
m = Math.floor(totalSeconds / 60)
s = totalSeconds % 60
return padZero(m) + ':' + padZero(s)
```

padZero(n):

```
return String(n).padStart(2, '0')
```

6. 存储设计

6.1 本地存储键值

键名	存储内容	说明
selectedDeviceId	设备标识字符串	当前选中设备, 编码后存储
pendingAlarm	报警数据对象	跨页传递的待处理报警

6.2 存储使用说明

- **selectedDeviceId**: 首页点击设备后设置，控制页 onShow 时读取
- **pendingAlarm**: 首页触发模拟报警时设置，控制页读取后立即清除
- 所有存储使用 encodeURIComponent 编码特殊字符

7. 事件通信设计

7.1 全局事件

事件名	参数	触发方	接收方	说明
deviceAlarm	{deviceId, deviceName, alarmCmd, alarmLabel, timestamp}	控制页/首页	首页	异常上报时通知 首页跳转

7.2 事件使用方式

```
// 发送事件
uni.$emit('deviceAlarm', { ... })

// 监听事件
uni.$on('deviceAlarm', handler)

// 取消监听
uni.$off('deviceAlarm', handler)
```

文档版本： V1.0 编写日期： 2026 年 6 月 28 日 审核状态： 待审核